分数 6 程序填空题 作者 李松 单位 浙江大学

Given the head of a singly linked list, swap every two adjacent nodes if the value of the first node is greater than the value of the second node and the value of the first node is even. If the number of nodes is odd, the last node remains unchanged. Complete the two blanks in the code below.

```
#include <stdio.h>
#include <stdlib.h>
struct ListNode {
    int val;
    struct ListNode *next;
};
struct ListNode* swapIfGreaterAndEven(struct ListNode* head) {
    struct ListNode dummy;
    dummy.next = head;
    struct ListNode *prev = &dummy;

    while (prev->next != NULL && prev->next->next != NULL) {
        struct ListNode *first = prev->next;
        struct ListNode *second = first->next;

        if ( first->val > second->val && first->val % 2 == 0 | 3 分 ) {
            first->next = first->next->next | 3 分 ;
            second->next = first;

            prev->next = second;
            prev = first;
        } else {
            prev = second;
        }
    }
    return dummy.next;
}
```

评测结果 答案正确

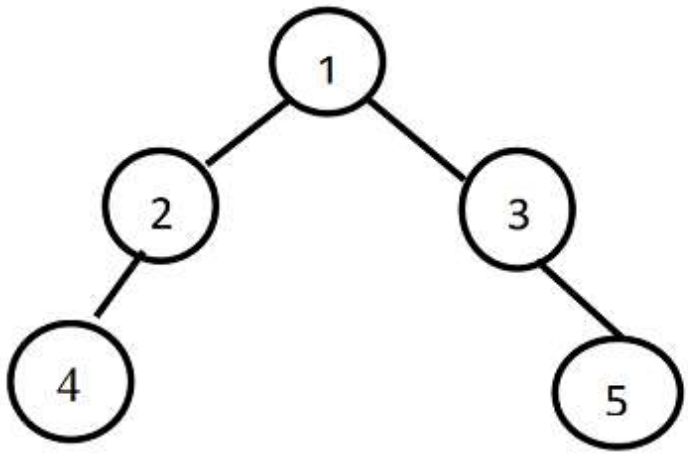
得分 6 分

R5-2 分数 6 Find the first node with a right child (2) 作者 何钦铭 单位 浙江大学

In a binary tree represented as a complete binary tree, find the first node with a right child encountered during an in-order traversal. If no such node exists, return -1; otherwise, return that node's value.

For example: a binary tree stored in this format (as illustrated in the figure) has its root node at array index 1, with missing nodes represented by 0 in the array.

Please complete the function **int FirstHaveRight(int a[], int n)**, where parameter **n** is the index of the last element in array **a** (e.g., **n** = 7 in the provided example). All elements after the last element of array **a** are initialized to 0. The algorithm in this function is similar to iterative implementation of in-order traversal.



数组 a	--	1	2	3	4	0	0	5	
下标	0	1	2	3	4	5	6	7	

```
int FirstHaveRight(int a[],int n)
{  int stack[MAXSIZE];
   int top=-1, tree=1,treeRight;

   for ( ; ; ) {
     for ( ; tree<=n; tree = tree*2 ){   (3 分)
         if (a[tree]!=0) stack[++top]=tree;
         else break;
     }
     if (top==-1) return -1;
     else tree = stack[top--];
     treeRight=2*tree+1;
     if ( a[treeRight] != 0 ) break;   (3 分)
     else tree = treeRight;
   }
   return a[tree];
}
```

评测结果 答案正确

得分 6 分